Memorial Descritivo - Arquitetura e Organização de Computadores

Disciplina: Arquitetura e Organização de Computadores

Semestre: 2025/02

Docente: Cláudio Camargo Rodrigues

Desafios de Programação em MIPS Assembly

1. Introdução

- **Objetivo do Desafio:** O presente trabalho visa a aplicação prática dos conceitos de Arquitetura de Computadores através da programação em linguagem de montagem (Assembly) para a arquitetura MIPS. O foco é compreender o funcionamento de um processador RISC, a manipulação de registradores, estágios de pipeline e a tradução de algoritmos de alto nível para instruções de baixo nível.
- **Descrição do Problema:** Foram propostos cinco problemas distintos que abrangem desde lógica aritmética básica e manipulação de vetores até aplicações complexas de engenharia, como o cálculo de balanço hídrico em redes de distribuição (usando matrizes) e a cinemática inversa para o controle de um braço robótico 2D (usando Séries de Taylor para funções transcendentes).

2. Estrutura do Grupo

- **Integrantes do Grupo:**
  - Leonardo Guimarães Abreu
  - Diogo Ferraz de Siqueira
  - Cauê Fernando Faria de Abreu

- **Divisão de Tarefas:**

A princípio, foi dividido entre os membros do grupo que Diogo e Cauê resolveriam dois exercícios cada, e Leonardo faria apenas um e mais a documentação final do trabalho. Além disso, os exercícios que cada um deveria fazer foram sorteados aleatoriamente. Logo, resumiu-se da seguinte forma a divisão de tarefas:

- **Leonardo:** Desenvolvimento da biblioteca de funções trigonométricas inversas (Exercício 4) via Séries de Taylor e consolidação da documentação final do trabalho nas normas exigidas.
- **Diogo:** Implementação dos algoritmos de potenciação (Exercício 1) e do filtro de classificação de números pares e ímpares em vetores (Exercício 2).
- **Cauê:** Implementação do sistema de diagnóstico de redes via matrizes de incidência (Exercício 3) e do controlador de cinemática inversa para o braço robótico (Exercício 5).

3. Soluções Propostas

- **Descrição da Solução:**
  - **Exercício 1 (Potenciação):** Implementado através de um laço iterativo que realiza multiplicações sucessivas da base pelo número de vezes definido no expoente.
  - **Exercício 2 (Par/Ímpar):** Utiliza a instrução `div` para obter o resto da divisão (`mfhi`) por 2, direcionando o fluxo do programa para armazenar o valor no vetor correspondente (pares ou ímpares) na memória.
  - **Exercício 3 (Fluxo de Água):** A solução realiza a multiplicação de uma matriz de incidência n x m por um vetor de fluxo m x 1. O resultado determina se cada nó é uma fonte (xi < 0), sumidouro (xi > 0) ou está em equilíbrio (xi = 0).
  - **Exercício 4 (Séries de Taylor):** Implementação de aproximações polinomiais para `arcsin(x)`, `arccos(x)` e `arctan(x)`. Para evitar *overflow* de fatoriais, os termos foram calculados iterativamente com base no termo anterior, utilizando 20 iterações para garantir precisão.
  - **Exercício 5 (Braço Robótico):** Esta solução implementa o controle de "um manipulador robótico de dois elos através da cinemática inversa. O algoritmo utiliza a **Lei dos Cossenos** para calcular os ângulos das juntas ($\theta 1$ e $\theta 2$) necessários para que a ponta do braço alcance uma coordenada específica (Px ,Py) no plano 2D.

- **Tecnologias Utilizadas:** Simulador MARS (MIPS Assembler and Runtime Simulator).
- **Arquitetura do Sistema:** O sistema baseia-se na arquitetura MIPS, um modelo RISC que utiliza carregamento/armazenamento (*Load/Store*), instruções de tamanho fixo (32 bits) e uma filosofia de conjunto de instruções reduzido para otimizar o desempenho do pipeline.

4. Estratégias Utilizadas

- **Metodologia de Desenvolvimento:** Adotou-se o desenvolvimento modular. Cada integrante focou em problemas específicos, testando a lógica individualmente antes da integração no memorial descritivo.
- **Planejamento e Cronograma:** Durante todo o mês de janeiro, os integrantes deveriam desenvolver a resolução dos exercícios propostos e enviá-los para Leonardo. Após esse período, esse ficaria encarregado de realizar a documentação do trabalho para a entrega final, utilizando desde o início do mês de fevereiro até o seu fim para tanto.
- **Gestão de Riscos:** O principal risco identificado foi o estouro de memória (*overflow*) em cálculos matemáticos (Exercício 4) e erros de endereçamento de memória em vetores (Exercícios 2 e 3). Para mitigá-los, utilizou-se aritmética de ponto flutuante e cálculos iterativos.

5. Testes Realizados

- **Plano de Testes:** Foram utilizados valores de entrada conhecidos para validar as saídas. No exercício 4, valores entre -1.0 e 1.0 foram testados contra calculadoras científicas. No exercício 3, utilizou-se o grafo e fluxos fornecidos no roteiro.
- **Tipos de Testes:** Testes unitários para cada instrução lógica e testes de sistema para verificar a saída final no console do simulador.
- **Resultados dos Testes:** Os programas apresentaram comportamento estável. O diagnóstico de rede (Exercício 3) identificou corretamente as fontes e sumidouros, e as funções trigonométricas (Exercício 4) atingiram a precisão esperada.

6. Experiência Vivenciada

- **Desafios Enfrentados:** A maior dificuldade foi a manipulação de registradores de ponto flutuante (`$f0`, `$f12`, etc.) e a implementação de saltos condicionais complexos em substituição às estruturas `if/else` de alto nível.
- **Lições Aprendidas:** Compreensão profunda de como o hardware gerencia dados na memória e a importância de algoritmos eficientes (como a redução de complexidade na exponenciação) para o desempenho da CPU.

7. Conclusão

O trabalho permitiu consolidar os conhecimentos de AOC, demonstrando que mesmo problemas complexos de engenharia e física podem ser resolvidos em nível de

máquina através de uma lógica rigorosa e do uso eficiente dos recursos de um processador RISC.

8. Anexos (Código Fonte)

P1: Potenciação (potencia.asm)

Programa: Algoritmos de Exponenciação (Simples e Quadrática).

Descrição: Implementação de dois métodos para o cálculo de x^n. O primeiro segue uma lógica linear $O(n)$, enquanto o segundo utiliza a técnica de exponenciação quadrática para atingir complexidade logarítmica $O(\log n)$.

Arquivo: potencia.asm.

```
==================================================================

.data
   base:   .word 2
   expo:   .word 10
   res1:   .word 0
   res2:   .word 0
   msg1:   .asciiz "Resultado Algoritmo 1 (O(n)): "
   msg2:   .asciiz "\nResultado Algoritmo 2 (O(log n)): "

.text
main:
   # --- ALGORITMO 1 (Fragmento 1 - Simples) ---
   lw $t0, base
   lw $t1, expo
   li $t2, 1        # result = 1

loop_simples:
```

```
    blez $t1, fim_simples
    mul $t2, $t2, $t0   # result *= x
    addi $t1, $t1, -1   # n--
    j loop_simples

fim_simples:
    sw $t2, res1

    # --- ALGORITMO 2 (Fragmento 2 - Quadrático) ---
    lw $t0, base       # x
    lw $t1, expo       # n
    li $t2, 1          # result = 1

loop_quadratico:
    beqz $t1, fim_quadratico

    # if (n % 2 == 1)
    andi $t3, $t1, 1    # verifica se ímpar usando bitwise AND
    beqz $t3, skip_mul
    mul $t2, $t2, $t0   # result *= x

skip_mul:
    srl $t1, $t1, 1     # n /= 2 (shift right)
    mul $t0, $t0, $t0   # x *= x
    j loop_quadratico

fim_quadratico:
    sw $t2, res2
```

```
    # Encerrar
    li $v0, 10
    Syscall

# --- ANÁLISE DE DESEMPENHO ---
# Algoritmo 1: Executa 'n' iterações. Para n=1000, executa ~4000 instruções.
# Algoritmo 2: Executa 'log2(n)' iterações. Para n=1000, executa ~60 instruções.
# O Algoritmo 2 é drasticamente mais rápido para valores grandes de n.
```

P2: Classificação Par/Ímpar (par-impar.asm)

P2: Filtro Par/Ímpar (par-impar.asm)

Programa: Classificação de Inteiros em Vetores.

Descrição: O algoritmo percorre um vetor V de inteiros armazenados em memória e, através do cálculo de paridade, separa os valores em dois novos vetores: "pares" e "impares".

Arquivo: par-impar.asm.

==========

```
# Programa: Filtro de Paridade em Vetores
# Arquivo: par-impar.asm

.data
    size:    .word 10
```

```
    vetor:   .word 15, 22, 30, 41, 55, 60, 78, 81, 90, 100
    pares:   .space 40   # espaço para 10 inteiros
    impares: .space 40

.text
main:
    la $s0, vetor      # Ponteiro V
    la $s1, pares      # Ponteiro pares
    la $s2, impares    # Ponteiro impares

    li $t0, 0          # i = 0
    lw $t1, size       # limite do loop

loop_principal:
    beq $t0, $t1, encerrar

    lw $t2, 0($s0)     # V[i]

    li $t3, 2
    div $t2, $t3
    mfhi $t4           # resto da divisão por 2

    beq $t4, $zero, eh_par

    # É Ímpar
    sw $t2, 0($s2)
    addi $s2, $s2, 4
    j proximo
```

```
eh_par:
   sw $t2, 0($s1)
   addi $s1, $s1, 4

proximo:
   addi $s0, $s0, 4
   addi $t0, $t0, 1
   j loop_principal

encerrar:
   li $v0, 10
   syscall
```

P3: Diagnóstico de Fluxo (fluxo-agua.asm)

Programa: Diagnóstico de Redes (Sistema de Distribuição de Água).

Descrição: Realiza o processamento matricial do produto entre uma Matriz de Incidência B e um vetor de fluxos f. O resultado x determina se cada nó da rede atua como fonte, sumidouro ou está em equilíbrio.

Arquivo: fluxo-agua.asm.

==========

```
# Programa: Diagnostico de Redes (Sistema de Distribuicao de Agua)
# Descrição: Calcula x = B . f e realiza diagnostico de cada no
# Arquivo: fluxo-agua.asm
```

.data # Matriz B - Matriz de Incidência (5 nós x 7 arestas) # Reflete o grafo: N0 (Fonte), N1 (ET1), N2 (ET2), N3 (Bairro A), N4 (Bairro B) matrizB: .word -1, -1, 0, 0, 0, 0, 0 # Nó 0 .word 1, 0, -1, -1, 0, 0, 0 # Nó 1 .word 0, 1, 1, 0, -1, -1, 0 # Nó 2 .word 0, 0, 0, 1, 1, 0, -1 # Nó 3 .word 0, 0, 0, 0, 0, 1, 1 # Nó 4

Vetor f - Caudal/Fluxo das arestas (A0 até A6) em L/s
vetorF: .word 100, 50, 80, 30, 80, 20, 10

Vetor x - Resultado do Balanço Líquido (B . f)
vetorX: .word 0, 0, 0, 0, 0

Mensagens para o console
msg_titulo: .asciiz "\n=== DIAGNOSTICO DE REDE DE AGUA ===\n"
msg_no: .asciiz "No "
msg_valor: .asciiz " = "
msg_ls: .asciiz " L/s -> "
msg_fonte: .asciiz "FONTE (Enviando)\n"
msg_sumidouro: .asciiz "SUMIDOURO (Recebendo)\n"
msg_equilibrio:.asciiz "PASSAGEM/EQUILIBRIO\n"
quebra: .asciiz "\n"

.text .globl main

main: # --- Passo 1: Calcular x = B . f (Multiplicação Matriz-Vetor) --- li $t0, 0 # i = indice da linha (nó)

loop_linha: bge $t0, 5, imprimir_resultados # Processar os 5 nós

li $t1, 0 # j = indice da coluna (aresta)
li $t2, 0 # soma acumulada para x[i]

loop_coluna: bge $t1, 7, salvar_x # Processar as 7 arestas

# Calcular offset da Matriz B: (i * num_colunas + j) * 4
mul $t3, $t0, 7
add $t3, $t3, $t1
sll $t3, $t3, 2 # t3 = t3 * 4

la $t4, matrizB
add $t4, $t4, $t3
lw $t5, 0($t4) # t5 = B[i][j]

```
# Calcular offset do Vetor f: j * 4
sll $t6, $t1, 2
la $t4, vetorF
add $t4, $t4, $t6
lw $t7, 0($t4)     # t7 = f[j]

# Multiplicação e Acúmulo
mul $t8, $t5, $t7   # t8 = B[i][j] * f[j]
add $t2, $t2, $t8   # soma += t8

addi $t1, $t1, 1
j loop_coluna


salvar_x: # Salvar resultado em vetorX[i] sll $t3, $t0, 2 la $t4, vetorX add $t4, $t4, $t3 sw
$t2, 0($t4)

addi $t0, $t0, 1
j loop_linha

# --- Passo 2: Imprimir Diagnóstico ---


imprimir_resultados: li $v0, 4 la $a0, msg_titulo syscall

li $t0, 0         # i = 0


loop_diag: bge $t0, 5, encerrar

# Imprimir "No X = "
li $v0, 4
la $a0, msg_no
syscall

li $v0, 1
move $a0, $t0
syscall

li $v0, 4
la $a0, msg_valor
```

```
syscall

# Carregar x[i] e imprimir valor
sll $t1, $t0, 2
la $t2, vetorX
add $t2, $t2, $t1
lw $t3, 0($t2)     # t3 = balanço líquido

li $v0, 1
move $a0, $t3
syscall

li $v0, 4
la $a0, msg_ls
syscall

# --- Classificação (Lógica do Pós-processamento) ---
beq $t3, $zero, eh_equilibrio
bgtz $t3, eh_sumidouro


eh_fonte: li $v0, 4 la $a0, msg_fonte syscall j prox_no

eh_sumidouro: li $v0, 4 la $a0, msg_sumidouro syscall j prox_no

eh_equilibrio: li $v0, 4 la $a0, msg_equilibrio syscall

prox_no: addi $t0, $t0, 1 j loop_diag

encerrar: li $v0, 10 syscall
```

P4: Cálculo de Funções Transcendentes (trig-1.asm)

Programa: Desvendando a Calculadora (Implementação de Séries de Taylor)

Descrição: Implementa as funções trigonométricas inversas (asin, acos, atan) utilizando expansões polinomiais de Taylor/Maclaurin.

Arquivo: trig-1.asm

==========

```
#Programa: Implementação de Séries de Taylor (asin, acos, atan)

#Descrição: Calcula aproximações para funções trigonométricas inversas

#==================================================================

.data # Strings de interface prompt_x: .asciiz "Digite o valor de x (entre -1.0 e 1.0): "
str_asin: .asciiz "\nArcsin(x): " str_acos: .asciiz "\nArccos(x): " str_atan: .asciiz
"\nArctan(x): "

# Constantes de Ponto Flutuante
c_zero:    .float 0.0
c_one:     .float 1.0
c_two:     .float 2.0
c_pi_2:    .float 1.57079632679  # Pi / 2 [cite: 159]

# Critério de Parada
iteracoes:  .word 20          # K = 20 iterações para precisão [cite: 161]


.text .globl main

main: # 1. Solicitar entrada li $v0, 4 la $a0, prompt_x syscall

# 2. Ler float x
li $v0, 6
syscall
mov.s $f12, $f0
mov.s $f20, $f12    # Salvar x original em $f20

# --- CALCULAR ARCTAN ---
jal arctan

li $v0, 4
la $a0, str_atan
```

```
syscall

mov.s $f12, $f0
li $v0, 2
syscall

# --- CALCULAR ARCSIN ---
mov.s $f12, $f20
jal arcsin

li $v0, 4
la $a0, str_asin
syscall

mov.s $f12, $f0
li $v0, 2
syscall

mov.s $f21, $f0     # Salvar arcsin(x) para o cálculo de acos

# --- CALCULAR ARCCOS ---
# Arccos(x) = Pi/2 - Arcsin(x) [cite: 159]
l.s $f1, c_pi_2
sub.s $f0, $f1, $f21

li $v0, 4
la $a0, str_acos
syscall

mov.s $f12, $f0
li $v0, 2
syscall

# Finalizar
li $v0, 10
syscall


#-------------------------------------------------------------------------------

#Função: arctan(x) [cite: 154, 155]
```

#Fórmula: x - x^3/3 + x^5/5 - x^7/7 ...

#----------

arctan: mov.s $f0, $f12 # Soma inicial = x mov.s $f1, $f12 # Potência atual mul.s $f2, $f12, $f12 # x^2

```
l.s   $f3, c_one     # Denominador
l.s   $f4, c_two     # Incremento do denominador

lw    $t0, iteracoes
li    $t1, 1         # Alternador de sinal
```

loop_atan: beq $t0, $zero, end_atan mul.s $f1, $f1, $f2 # x^(n+2) add.s $f3, $f3, $f4 # n + 2 div.s $f6, $f1, $f3 # Termo atual

```
beq   $t1, $zero, somar_atan
sub.s $f0, $f0, $f6
li    $t1, 0
j     next_iter_atan
```

somar_atan: add.s $f0, $f0, $f6 li $t1, 1 next_iter_atan: sub $t0, $t0, 1 j loop_atan end_atan: jr $ra

#----------

#Função: arcsin(x) [cite: 149, 150]

#Utiliza cálculo iterativo para evitar overflow de fatoriais [cite: 162, 163]

#----------

arcsin: mov.s $f0, $f12 # Soma = x mov.s $f1, $f12 # Potência = x mul.s $f2, $f12, $f12 # x^2 l.s $f3, c_one # Coeficiente acumulado l.s $f4, c_one # n = 1.0

```
l.s   $f10, c_one
l.s   $f11, c_two
lw    $t0, iteracoes
```

loop_asin: beq $t0, $zero, end_asin

Atualizar Coeficiente: C_n = C_{n-1} * (2n-1)/(2n)
mul.s $f6, $f4, $f11 # 2n

```
sub.s $f7, $f6, $f10  # 2n - 1
div.s $f8, $f7, $f6   # (2n-1)/2n
mul.s $f3, $f3, $f8   # C_n

mul.s $f1, $f1, $f2   # x^(2n+1)
add.s $f9, $f6, $f10  # 2n + 1

# Termo = (C_n * x^p) / (2n+1)
mul.s $f5, $f3, $f1
div.s $f5, $f5, $f9

add.s $f0, $f0, $f5   # Acumular na soma

add.s $f4, $f4, $f10  # n++
sub   $t0, $t0, 1
j    loop_asin


end_asin: jr $ra



.data # Strings de interface prompt_x: .asciiz "Digite o valor de x (entre -1.0 e 1.0): "
str_asin: .asciiz "\nArcsin(x): " str_acos: .asciiz "\nArccos(x): " str_atan: .asciiz
"\nArctan(x): " newline: .asciiz "\n"

# Constantes de Ponto Flutuante
c_zero:    .float 0.0
c_one:     .float 1.0
c_two:     .float 2.0
c_pi_2:    .float 1.57079632679  # Pi / 2

# Critério de Parada
iteracoes:  .word 20           # K = 20 iterações (precisão suficiente)


.text .globl main

main: # 1. Imprimir prompt li $v0, 4 la $a0, prompt_x syscall

# 2. Ler float x (o valor fica em $f0)
li $v0, 6
syscall
```

```
# Mover x de $f0 para $f12 (argumento padrão) para passar para as funções
mov.s $f12, $f0

# Salvar x em um registrador salvo ($f20) para usar nas próximas chamadas
mov.s $f20, $f12

# --- CALCULAR ARCTAN ---
# Argumento já está em $f12
jal arctan

# Imprimir label
li $v0, 4
la $a0, str_atan
syscall

# Imprimir resultado ($f0)
mov.s $f12, $f0  # syscall 2 imprime o que está em $f12
li $v0, 2
syscall

# --- CALCULAR ARCSIN ---
# Restaurar x original para $f12
mov.s $f12, $f20
jal arcsin

# Imprimir label
li $v0, 4
la $a0, str_asin
syscall

# Imprimir resultado
mov.s $f12, $f0
li $v0, 2
syscall

# Salvar resultado de arcsin em $f21 para usar no arccos
mov.s $f21, $f0

# --- CALCULAR ARCCOS ---
# Arccos(x) = Pi/2 - Arcsin(x)
# Não precisamos de função complexa, apenas subtração
```

Mas se fosse uma função separada, passaríamos x e chamaríamos arcsin dentro dela.

```
# Carregar PI/2
l.s $f1, c_pi_2

# Calcular: PI/2 - arcsin(x)
sub.s $f0, $f1, $f21

# Imprimir label
li $v0, 4
la $a0, str_acos
syscall

# Imprimir resultado
mov.s $f12, $f0
li $v0, 2
syscall

# --- FIM ---
li $v0, 10
syscall
```

Função: arctan(x)

Entrada: $f12 (x)

Saída: $f0 (resultado)

Fórmula: x - x^3/3 + x^5/5 - x^7/7 ...

---

arctan: # Inicialização mov.s $f0, $f12 # Soma = x (primeiro termo) mov.s $f1, $f12 # Potência atual (x^n) = x mul.s $f2, $f12, $f12 # x_quadrado = x * x (para avançar potência de 2 em 2)

```
l.s   $f3, c_one     # Denominador atual (1, 3, 5...)
l.s   $f4, c_two     # Constante 2.0 para incrementar denominador
l.s   $f5, c_zero    # Usado para inverter sinal (0 - termo)

lw    $t0, iteracoes  # Contador de loop
```

li $t1, 1 # Flag de sinal (1 = subtrair, -1 = somar na prox)

loop_atan: beq $t0, $zero, end_atan

1. Atualizar Potência: x^(n+2) = x^n * x^2
mul.s $f1, $f1, $f2

2. Atualizar Denominador: n = n + 2
add.s $f3, $f3, $f4

3. Calcular Termo: (x^pot) / den
div.s $f6, $f1, $f3

4. Aplicar sinal alternado
Se $t1 == 1, subtraímos. Se $t1 == 0, somamos.
beq $t1, $zero, somar_atan

subtrair_atan:

sub.s $f0, $f0, $f6 # soma = soma - termo li $t1, 0 # Próximo será soma j next_iter_atan

somar_atan:

add.s $f0, $f0, $f6 # soma = soma + termo li $t1, 1 # Próximo será subtração

next_iter_atan:

sub $t0, $t0, 1 # Decrementa contador j loop_atan

end_atan:

jr $ra

Função: arcsin(x)

Entrada: $f12 (x)

Saída: $f0 (resultado)

Fórmula: x + (1/2)(x^3/3) + (13/24)(x^5/5) ...

Termo Geral Iterativo: T_n = T_{n-1} * (2n-1)/(2n) * x^2 * (2n-1)/(2n+1) ??

Simplificação: Manter o coeficiente separado da potência/divisor.

Coeficiente C_n = C_{n-1} * (2n-1)/(2n).

Termo da soma = C_n * (x^(2n+1)) / (2n+1)

---

arcsin: # Registradores: # $f0 = Soma total # $f1 = Potência de x (x, x^3, x^5...) # $f2 = x^2 (constante multiplicadora da potência) # $f3 = Coeficiente acumulado (1, 1/2, 3/8...) # $f4 = Contador n (ponto flutuante: 1.0, 2.0...) # $f5 = Auxiliares

```
mov.s $f0, $f12      # Soma = x
mov.s $f1, $f12      # Potência = x
mul.s $f2, $f12, $f12 # x^2
l.s   $f3, c_one     # Coeficiente inicial = 1.0
l.s   $f4, c_one     # n = 1.0

l.s   $f10, c_one    # Constante 1.0
l.s   $f11, c_two    # Constante 2.0

lw    $t0, iteracoes  # Loop K vezes


loop_asin: beq $t0, $zero, end_asin

# --- Passo 1: Atualizar Coeficiente C_n ---
# Fator = (2n - 1) / (2n)

# $f6 = 2n
mul.s $f6, $f4, $f11

# $f7 = 2n - 1
sub.s $f7, $f6, $f10

# Fator = (2n-1) / 2n
div.s $f8, $f7, $f6

# Novo Coeficiente = Antigo * Fator
mul.s $f3, $f3, $f8

# --- Passo 2: Atualizar Potência x^(2n+1) ---
mul.s $f1, $f1, $f2

# --- Passo 3: Calcular Termo Atual ---
# Divisor da série = 2n + 1
```

```
add.s $f9, $f6, $f10  # f9 = 2n + 1

# Termo = (Coeficiente * Potência) / (2n + 1)
mul.s $f_temp, $f3, $f1
div.s $f_temp, $f_temp, $f9

# --- Passo 4: Somar ao total ---
add.s $f0, $f0, $f_temp

# Preparar próxima iteração
add.s $f4, $f4, $f10  # n = n + 1
sub   $t0, $t0, 1
j    loop_asin


end_asin:

        jr $ra
```

P5: Controlador de Braço Robótico 2D (robot2D.asm)

Programa: Controlador de Braço Robótico 2D (Cinemática Inversa)

Descrição: Desenvolve um módulo para determinar as configurações das juntas (θ1 ,θ2) necessárias para alcançar uma posição (x,y) específica no plano de trabalho. Utiliza a Lei dos Cossenos e as funções trigonométricas implementadas via Séries de Taylor para calcular os ângulos de controle.

Arquivo: robot2D.asm

```
==================================================================

#Programa: Controlador de Braço Robótico 2D (Cinemática Inversa)

#Descrição: Calcula os ângulos theta1 e theta2 dado Px, Py, L1, L2

#==================================================================

#==================================================================

.data
```

```
 prompt_px: .asciiz "Digite Px (coordenada X do end-effector): "

 prompt_py: .asciiz "Digite Py (coordenada Y do end-effector): "

prompt_l1: .asciiz "Digite L1 (comprimento do segmento 1): "

prompt_l2:

        .asciiz "Digite L2 (comprimento do segmento 2): "

str_theta1:

        .asciiz "\nTheta1 (graus): "
str_theta2:

         .asciiz "\nTheta2 (graus): "
str_unreachable:

        .asciiz "\nPonto FORA DE ALCANCE do robo!\n"
newline:

        .asciiz "\n"

c_zero:

        .float 0.0
c_one:

        .float 1.0
c_two:

        .float 2.0
c_pi_2:

        .float 1.57079632679   # Pi/2
c_pi:

        .float 3.14159265358   # Pi
c_rad2deg:

        .float 57.2957795131   # 180 / Pi

iteracoes:

        .word 20


.text
```

```
.globl main

main:

# --- Leituras de Entrada ---

li $v0, 4 la $a0, prompt_px

syscall

li $v0, 6

syscall

mov.s $f20, $f0 # Px

li $v0, 4
la $a0, prompt_py
syscall
li $v0, 6
syscall
mov.s $f21, $f0        # Py

li $v0, 4
la $a0, prompt_l1
syscall
li $v0, 6
syscall
mov.s $f22, $f0        # L1

li $v0, 4
la $a0, prompt_l2
syscall
li $v0, 6
syscall
mov.s $f23, $f0        # L2

# Passo 1: Calcular D^2 = Px^2 + Py^2
mul.s $f1, $f20, $f20    # Px^2
mul.s $f2, $f21, $f21    # Py^2
add.s $f24, $f1, $f2     # $f24 = D^2
sqrt.s $f25, $f24       # $f25 = D
```

```
# Passo 2: Verificacao de Alcance
add.s $f3, $f22, $f23    # $f3 = L1 + L2
sub.s $f4, $f22, $f23    # $f4 = L1 - L2
abs.s $f4, $f4          # $f4 = |L1 - L2|

c.gt.s $f25, $f3        # D > L1 + L2?
bc1t out_of_range
c.lt.s $f25, $f4        # D < |L1 - L2|?
bc1t out_of_range

# Passo 3: Calcular theta2 (Lei dos Cossenos)
mul.s $f5, $f22, $f22    # L1^2
mul.s $f6, $f23, $f23    # L2^2
sub.s $f7, $f24, $f5
sub.s $f7, $f7, $f6      # $f7 = D^2 - L1^2 - L2^2
l.s $f8, c_two
mul.s $f8, $f8, $f22
mul.s $f8, $f8, $f23     # $f8 = 2*L1*L2
div.s $f12, $f7, $f8
jal arccos
mov.s $f26, $f0         # $f26 = theta2

# Passo 4: Calcular theta1
mov.s $f12, $f20        # Px
mov.s $f13, $f21        # Py
jal atan2_func
mov.s $f27, $f0         # $f27 = beta

add.s $f9, $f24, $f5     # D^2 + L1^2
sub.s $f9, $f9, $f6      # D^2 + L1^2 - L2^2
l.s $f10, c_two
mul.s $f10, $f10, $f25   # 2*D
mul.s $f10, $f10, $f22   # 2*D*L1
div.s $f12, $f9, $f10
jal arccos
mov.s $f28, $f0         # $f28 = phi
sub.s $f29, $f27, $f28   # theta1 = beta - phi

# Passo 5: Conversao e Saida
l.s $f30, c_rad2deg
li $v0, 4
```

```
la $a0, str_theta1
syscall
mul.s $f12, $f29, $f30   # theta1 graus
li $v0, 2
syscall
li $v0, 4
la $a0, str_theta2
syscall
mul.s $f12, $f26, $f30   # theta2 graus
li $v0, 2
syscall
j fim


out_of_range:

        li $v0, 4 la $a0, str_unreachable syscall

fim:

        li $v0, 10

        syscall

#--- Sub-funcoes de Calculo ---

arccos:

        sub $sp, $sp, 4

        sw $ra, 0($sp)

        jal arcsin

        l.s $f1, c_pi_2

        sub.s $f0, $f1, $f0

        lw $ra, 0($sp)

        add $sp, $sp, 4

        jr $ra

arcsin:
```

```
        mov.s $f0, $f12
        mov.s $f1, $f12
        mul.s $f2, $f12, $f12
        l.s $f3, c_one
        l.s $f4, c_one
        l.s $f10, c_one
        l.s $f11, c_two
        lw $t0, iteracoes
loop_asin:
        beq $t0, $zero, end_asin
        mul.s $f6, $f4, $f11
        sub.s $f7, $f6, $f10
        div.s $f8, $f7, $f6
        mul.s $f3, $f3, $f8
        mul.s $f1, $f1, $f2
        add.s $f9, $f6, $f10
        mul.s $f15, $f3, $f1
        div.s $f15, $f15, $f9
        add.s $f0, $f0, $f15
        add.s $f4, $f4, $f10
        addi $t0, $t0, -1
        j loop_asin
end_asin:
        jr $ra
```

```
arctan:
        mov.s $f0, $f12
        mov.s $f1, $f12
        mul.s $f2, $f12, $f12
        l.s $f3, c_one
        l.s $f4, c_two
        lw $t0, iteracoes
        li $t1, 1
loop_atan:
         beq $t0, $zero, end_atan
        mul.s $f1, $f1, $f2
        add.s $f3, $f3, $f4
        div.s $f6, $f1, $f3
        beq $t1, $zero, somar_atan
subtrair_atan:
        sub.s $f0, $f0, $f6
        li $t1, 0
        j prox_atan
        somar_atan: add.s $f0, $f0, $f6
        li $t1, 1
prox_atan:
        addi $t0, $t0, -1
        j loop_atan
end_atan:
```

```
        jr $ra
atan2_func:
        sub $sp, $sp, 4
        sw $ra, 0($sp)
        l.s $f16, c_zero
        l.s $f17, c_pi
        l.s $f18, c_pi_2
        c.eq.s $f12, $f16 bc1t atan2_x_zero
        div.s $f12, $f13, $f12
        jal arctan
        l.s $f16, c_zero
        c.lt.s $f20, $f16 bc1f atan2_done
        c.lt.s $f21, $f16 bc1t atan2_sub_pi
        add.s $f0, $f0, $f17
        j atan2_done
atan2_sub_pi:
        sub.s $f0, $f0, $f17
        j atan2_done
atan2_x_zero:
        c.lt.s $f16, $f13 bc1t atan2_pos_pi2
        c.lt.s $f13, $f16 bc1t atan2_neg_pi2
        mov.s $f0, $f16
        j atan2_done atan2_pos_pi2:
        mov.s $f0, $f18
```

```
        j atan2_done
atan2_neg_pi2:
        neg.s $f0, $f18
atan2_done: lw $ra, 0($sp)
        add $sp, $sp, 4 jr $ra
```